

Project Chronos: Ultimate Revision & Interview Guide

1. Project Overview & Problem Statement

- **What it is:** A Distributed Market Digital Twin. It simulates a live stock market where AI agents (Neural Networks) and algorithmic bots trade against each other in real-time.
- **Problem Solved:** Traditional algorithmic backtesting uses static historical data. If a bot places a huge order, the historical data doesn't react. Chronos solves this by creating a "living" market. Massive trades actually move the price, triggering panic or euphoria in other bots.

2. System Architecture

Chronos abandons traditional monolithic design for a **Distributed Microservice Architecture**.

- **The Server Node (True Engine):** Built in Python. Holds the Limit Order Book (LOB). Listens on TCP Port 5555.
- **The Client Node (Agent Swarm):** 100 concurrent AI agents requesting data and sending orders.
- **The Oracle Node:** Built with the Google Gemini API. Listens to breaking news, scores sentiment, and executes massive trades to shock the market.
- **The UI Bridge:** A Flask server and Socket.IO connection that pushes real-time market JSON data to a browser dashboard built with Lightweight Charts.

3. Technology Stack Deep Dive

- **Python (Core Engine):** Chosen because the ML agents were built in PyTorch. Using Python allowed seamless data integration. However, Python is slow for High-Frequency Trading due to the GIL (Global Interpreter Lock). We bypassed the GIL by running the Engine and the Agents in entirely separate processes.
- **ZeroMQ (Networking):** Chosen over HTTP/REST APIs. REST APIs have heavy HTTP headers causing latency. ZeroMQ uses raw, brokerless TCP sockets allowing lightning-fast, peer-to-peer data transfer (1,200+ requests/sec locally).
- **Flask & WebSockets (Socket.IO):** Chosen to stream data to the frontend. Standard HTTP requires the browser to constantly poll the server. WebSockets keep a persistent tunnel open, allowing the server to push data instantly at 60fps.
- **Lightweight Charts (Frontend):** Standard HTML/JS charting libraries (like Chart.js) use SVG graphics which lag under heavy data. Lightweight Charts uses HTML5 `<canvas>` (GPU rendering) to handle thousands of financial candles smoothly.
- **Google Gemini API (Macro Oracle):** Chosen over traditional NLP (like VADER). Traditional NLP relies on keyword matching and struggles with financial irony. Gemini (an LLM) understands complex financial context (e.g., "Company beats earnings but lowers guidance").

4. Complete Execution Workflow

1. **Initialization:** Engine, Swarm, Oracle, and UI run in separate terminals.
2. **User Request:** User clicks "Start". Browser sends a WebSocket event to Flask. Flask sends a ZeroMQ `INIT_SIM` command to the Engine.
3. **Market Loop:** 100 Agents constantly request the LOB state via ZeroMQ. They process the data through their Neural Networks and send `BUY` or `SELL` orders back.
4. **Execution:** The Engine matches buyers and sellers natively using Python dictionaries. It calculates the new price, updates cash balances, and prunes old orders.
5. **Data Broadcasting:** A background thread in Flask asks the Engine for state every second and emits it to the Browser via WebSocket.
6. **Macro Shock:** User injects news -> Oracle asks Gemini for a score (-1.0 to 1.0) -> Oracle calculates a massive dump/pump order -> Sends to Engine -> Engine clears the book and gaps the price.

5. Technical Q&A

Q: How does the Limit Order Book actually work?

A: It is implemented as pure Python Dictionaries (`self.bids` and `self.asks`). The keys are the prices, and the values are lists of orders at that price. We chose dictionaries over SQL databases because RAM lookup is $O(1)$ time complexity, whereas database Disk I/O takes milliseconds (too slow for HFT).

Q: What happens if an Agent's Limit Order doesn't match instantly?

A: It sits in the dictionary. However, the engine has a pruning mechanism (`apply_market_regime_physics`). If the market price drifts \$0.50 away from the order, the engine deletes the order to prevent RAM bloat and keep lookups fast.

Q: How does the Oracle decide how much stock to buy/sell?

A: Gemini outputs a sentiment score (-1.0 to 1.0). The Oracle targets a max 20% price gap. It uses the formula: $\text{Required Qty} = (\text{Target Gap} / \$2.00) * 100,000$. It ensures a minimum of 50,001 shares are traded to trigger a Black Swan event in the engine.

Q: What happens if an agent crashes? Does the engine freeze?

A: No. The Engine implements `zmq.Poller` with a strict 50ms timeout. If an agent hangs, the engine skips it and moves to the next tick, preventing system deadlock.

Q: Who are the 100 agents?

A: 15 Market Makers (profit on the spread), 4 Institutional Whales (trade in massive blocks), and 80 Retail Traders (chaotic, emotional trading). If Retail Traders go bankrupt, they are instantly liquidated and respawned with fresh cash so the market doesn't die.